PTMI Supporting Report

Module: Mobile Web Application Development · Assessment 1: Programming for React · Student ID: 34149909

1. Design Justification

The Patient Treatment Management Interface (PTMI) is for clinic staff who often work on phones or tablets between appointments, so a **mobile-first** approach was used. The wireframe (Figure 1) was designed for a 390px-wide screen first and then scaled up to tablet and desktop layouts.

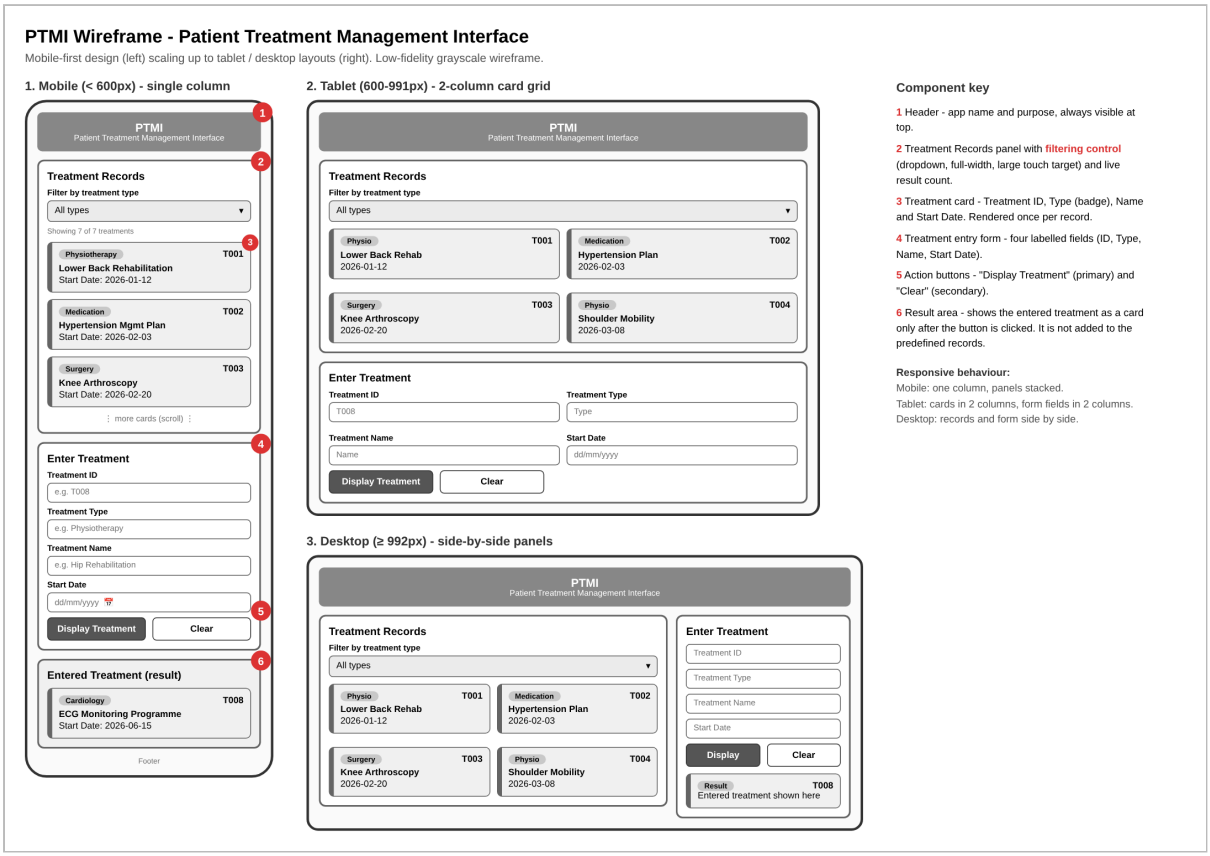


Figure 1: Final mobile-first PTMI wireframe (mobile, tablet and desktop layouts)

- **Single-column mobile layout:** Treatment records, filter and entry form are stacked vertically in the order staff need them (view, filter, then enter). This removes horizontal scrolling and keeps one clear reading path.
- **Card-based records:** Each treatment is a card showing Treatment ID, Type, Name and Start Date. Cards are easier to read on narrow screens than a wide table. A coloured type badge lets staff recognise the category at a glance.
- **Filtering control:** A single full-width dropdown sits directly above the records, so its effect is immediately visible. A live count (“Showing 2 of 7 treatments”) gives feedback, and an “All types” option resets the view.
- **Entry form:** Four clearly labelled fields with placeholder examples, a native date picker for Start Date, and a primary “Display Treatment” button next to a secondary “Clear” button. Validation tells the user if a field is empty.
- **Usability and accessibility:** Inputs and buttons are at least 44px tall for touch use; labels are linked to inputs; text/background contrast is high; and a consistent blue medical palette with a restrained style suits a clinical setting.
- **Responsive behaviour:** CSS media queries at 600px and 992px change the layout: one column on mobile, a two-column card grid on tablet, and records and form side by side on desktop (form panel stays visible while scrolling).

2. Development Progression and Evidence

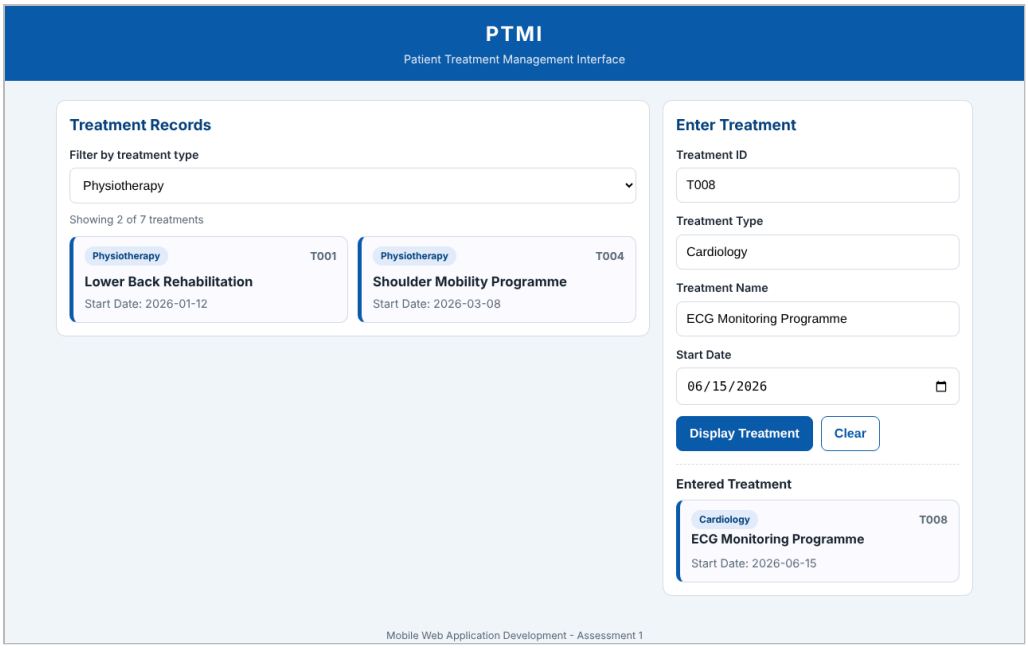


Figure 2a: Implemented React interface in a desktop browser (filter set to Physiotherapy, form result shown)

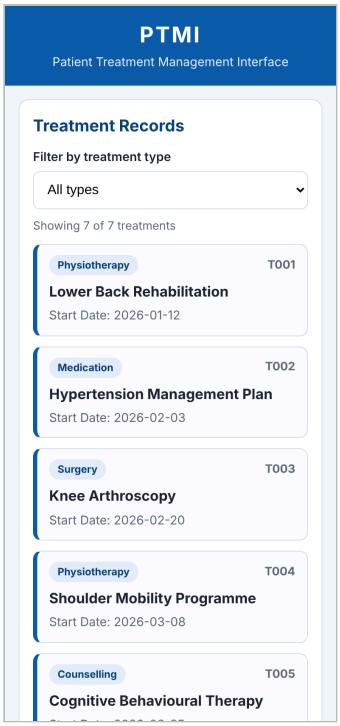


Figure 2b: Same app at mobile width (390px)

3. How the Wireframe Was Translated into the Application

The application was built with React (Vite) using functional components. Each numbered region of the wireframe became one component or one section of CSS:

Wireframe element	React implementation
1 Header	`App.jsx` header section
2 Records panel and filter	`FilterControl.jsx` (dropdown) and `TreatmentList.jsx` (list)
3 Treatment card	`TreatmentCard.jsx`, reused for every record and for the form result
4–6 Form, buttons, result	`TreatmentForm.jsx`
Predefined records	`data/treatments.js` (JSON-style array of 7 objects)

- **Components and props:** `App` holds the shared state and passes data down. `TreatmentList` receives the treatments array, and each `TreatmentCard` receives its four attributes as props. `FilterControl` receives the selected type and a change function.
- **JSON, arrays and .map():** The seven predefined records live in a local array of objects. `.map()` renders one card per record in `TreatmentList`, and also creates the dropdown options from the list of types in `FilterControl`.
- **useState and events:** `selectedType` in `App` is updated by the dropdown's `onChange`, and the list is filtered by that value. In the form, each of the four inputs has its own state variable that is updated by its `onChange` handler. The `onClick` of "Display Treatment" (`handleDisplay`) checks that no field is empty and stores the data in a separate `shownTreatment` state, so the entry is displayed without being added to the predefined records, as required. "Clear" resets the form.
- **Responsive CSS:** The base styles target mobile. `min-width` media queries then add the two-column grid (600px) and the side-by-side layout (992px), which is exactly the progression shown in the wireframe.
- **Code quality:** Components are in separate files with meaningful names (`treatmentType`, `filteredTreatments`, `handleDisplay`), data is separated from presentation, and unique `key` props are used in lists.

4. Testing and Differences from the Wireframe

The application was tested in Chrome at 390px and 1280px widths. Filtering to "Physiotherapy" correctly showed 2 of 7 records, "All types" showed all 7, and entering a treatment displayed it below the form while the records list stayed unchanged. One refinement was made during development: a validation message and a "Clear" button were added to the form to prevent empty submissions and make repeated entry quicker. Data is stored locally only, with no backend, REST API or database, as specified for Stage 1; this structure can be extended in Assessment 2.